LAB Manual – 3

Code/Course: 23CSE312/Distributed Systems

Topic: Video Streaming Service

[CO05] [BTL-3]
Student Name: D Ganeswara
Roll No.: AM.SC.U4CSE23119

Batch/Section: CSE-B
Date:13/02/2025

1 Aim

To design and implement a distributed video streaming application in Python that streams video data from a server to one or more clients and displays the video frames at the client side in real time.

2 Task

Develop a distributed application implementing a video streaming service using Python. A given video at the server side (for example, test.mp4) is streamed and played back at the client end. The client should display each video frame as soon as it is transmitted from the server.
The implementation requires the use of the OpenCV library for video opening and reading operations, and socket programming for transmitting video data between the server and the client.

3 Functional Requirements

The application should be capable of the following:

a. Providing streaming service to multiple clients. The server may be implemented either as an iterative server or a concurrent server.

b. Maintaining a repository of video files at the server side, from which the client can choose a video to stream.

Results

Server.py

```
import socket
import cv2
import threading
import os
import pickle
import struct
```

```python
HOST = '0.0.0.0'
PORT = 9999
VIDEO_DIR = "videos"

def handle_client(conn, addr):
    print(f"[+] Client connected: {addr}")

    try:
        videos = os.listdir(VIDEO_DIR)
        conn.sendall(pickle.dumps(videos))

        video_name = conn.recv(1024).decode()
        video_path = os.path.join(VIDEO_DIR, video_name)

        if not os.path.exists(video_path):
            print("[-] Invalid video requested")
            conn.close()
            return

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[-] Cannot open video")
            conn.close()
            return

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            _, buffer = cv2.imencode('.jpg', frame)
            data = pickle.dumps(buffer)

            message = struct.pack("Q", len(data)) + data
            conn.sendall(message)

        cap.release()

    except Exception as e:
        print(f"[!] Error: {e}")

    finally:
        conn.close()
        print(f"[-] Client disconnected: {addr}")

def main():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    server.bind((HOST, PORT))
    server.listen(5)

    print(f"[SERVER] Listening on {HOST}:{PORT}")

    while True:
        conn, addr = server.accept()
        thread = threading.Thread(target=handle_client, args=(conn, addr))
        thread.start()

if __name__ == "__main__":
    main()
```

Client.py

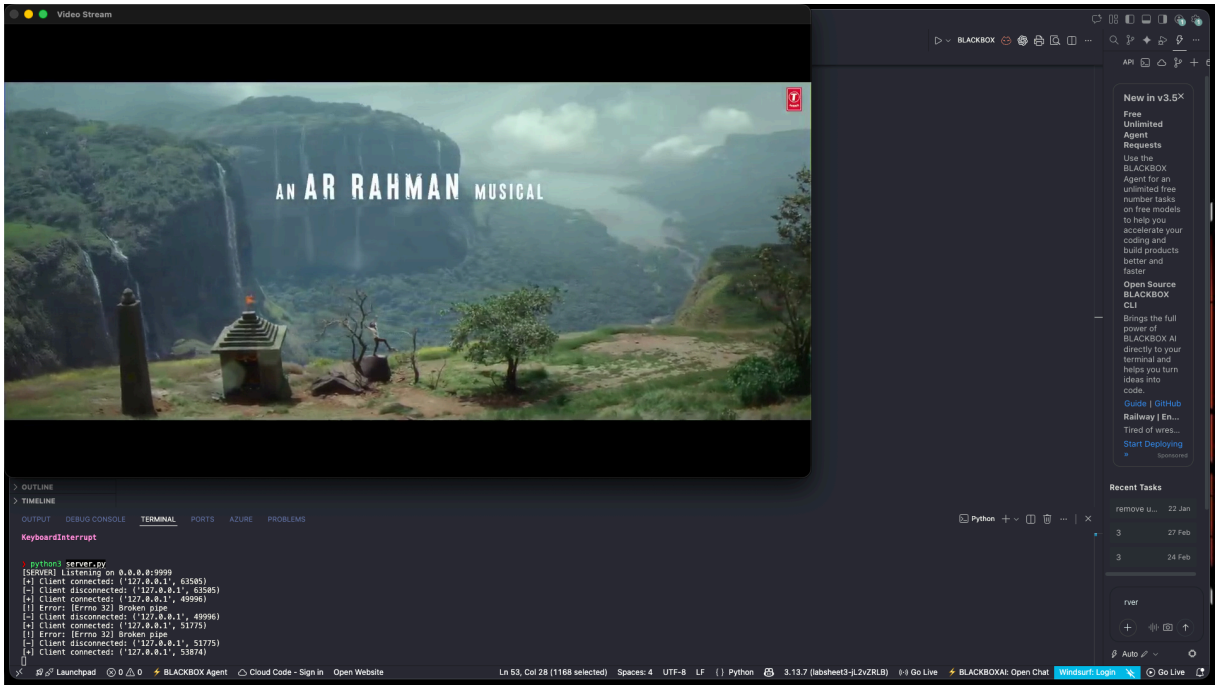
```
import socket
import cv2
import pickle
import struct

SERVER_IP = '127.0.0.1'
PORT = 9999

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))

data = b""
payload_size = struct.calcsize("Q")

videos = pickle.loads(client.recv(4096))
print("Available videos:")
for v in videos:
    print(" -", v)

choice = input("Enter video name: ")
client.send(choice.encode())

try:
    while True:
        while len(data) < payload_size:
            packet = client.recv(4096)
            if not packet:
                raise ConnectionError
            data += packet

        packed_size = data[:payload_size]
        data = data[payload_size:]
```

```
        frame_size = struct.unpack("Q", packed_size)[0]

        while len(data) < frame_size:
            data += client.recv(4096)

        frame_data = data[:frame_size]
        data = data[frame_size:]

        frame = pickle.loads(frame_data)
        frame = cv2.imdecode(frame, cv2.IMREAD_COLOR)

        cv2.imshow("Video Stream", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

except Exception:
    pass

finally:
    client.close()
    cv2.destroyAllWindows()
```

Screenshots

```
❯ python3 server.py
[SERVER] Listening on 0.0.0.0:9999
[+] Client connected: ('127.0.0.1', 63505)
[-] Client disconnected: ('127.0.0.1', 63505)
[+] Client connected: ('127.0.0.1', 49996)
[!] Error: [Errno 32] Broken pipe
[-] Client disconnected: ('127.0.0.1', 49996)
[+] Client connected: ('127.0.0.1', 51775)
[!] Error: [Errno 32] Broken pipe
[-] Client disconnected: ('127.0.0.1', 51775)
```

```
❯ python client.py
Available videos:
 - .DS_Store
 - test.mp4
 - Kalyani Official Lyrical Video.mp4
 - Chikiri Video Song.mp4
Enter video name: Kalyani Official Lyrical Video.mp4
```


Video Stream
AN AR RAHMAN MUSICAL
OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE PROBLEMS
KeyboardInterrupt
python3 server.py
[SERVER] Listening on 0.0.0.0:9999
[+] Client connected: ('127.0.0.1', 63505)
[-] Client disconnected: ('127.0.0.1', 63505)
[+] Client connected: ('127.0.0.1', 49996)
[!] Error: [Errno 32] Broken pipe
[-] Client disconnected: ('127.0.0.1', 49996)
[+] Client connected: ('127.0.0.1', 51775)
[!] Error: [Errno 32] Broken pipe
[-] Client disconnected: ('127.0.0.1', 51775)
[+] Client connected: ('127.0.0.1', 53874)